GUIA DE ESTUDIO

Como explicar el portfolio web

HTML, CSS, JavaScript, Node.js, Express, SQLite y comandos del proyecto.

> **Objetivo:** entender que hace cada parte y como se comunican para poder explicar el proyecto con tus propias palabras.

1. La idea general

El proyecto tiene un frontend y un backend. El frontend es lo que la persona ve y usa en el navegador; el backend recibe los mensajes y los guarda en la base de datos.

Formulario HTML → JavaScript valida y envia JSON → API de Express → Node.js valida → SQLite guarda el mensaje → el servidor responde

- **HTML:** organiza el contenido.
- **CSS:** define la apariencia y adaptacion a pantallas.
- **JavaScript del navegador:** agrega interactividad, valida el formulario y llama a la API.
- **Node.js y Express:** ejecutan el servidor y reciben solicitudes HTTP.
- **SQLite:** conserva los mensajes en un archivo de base de datos.

2. HTML: contenido y estructura

El archivo `index.html` define los elementos de la pagina. Las etiquetas semanticas describen el proposito de cada zona: `header` es el encabezado, `nav` la navegacion, `main` el contenido principal, `section` una seccion, `article` un contenido independiente y `footer` el pie.

- En `head` se configuran el idioma, la codificacion UTF-8, el viewport, el titulo y los enlaces al CSS y al JavaScript.
- Los enlaces como `href="#proyectos"` llevan a la seccion con el mismo `id`.
- Las habilidades usan `data-tooltip` para almacenar una descripcion que CSS muestra como ayuda visual.
- Las etiquetas `label` se conectan con los campos mediante `for` e `id`. Esto mejora la usabilidad y la accesibilidad.
- `required` indica que un campo es obligatorio. `maxlength` limita la cantidad de caracteres que puede escribir la persona.
- `novalidate` desactiva los mensajes nativos del navegador porque el proyecto presenta mensajes propios desde JavaScript.
- `role="status"` y `aria-live="polite"` ayudan a los lectores de pantalla a anunciar los cambios en el estado del envio.
- El atributo `defer` hace que el script se ejecute despues de que el HTML haya sido interpretado.

3. CSS: apariencia y adaptacion

El archivo `style/style.css` define colores, tipografia, espaciado, distribucion y estados visuales. Una regla CSS selecciona elementos y les aplica propiedades:

```
selector {
    propiedad: valor;
}
```

- `:root` contiene variables de colores como `--verde` y `--tinta`. Se usan con `var(--verde)` para mantener una paleta consistente.
- Un selector con `#`, por ejemplo `#contacto`, apunta a un ID. Un selector con `.`, como `.proyecto`, apunta a una clase.
- `display: flex` organiza elementos en una direccion flexible, como el titulo y el menu del encabezado.
- `display: grid` organiza contenido en filas y columnas, como las tarjetas de proyectos.
- `:hover` aplica estilos al pasar el cursor; `:focus-visible` ayuda a mostrar el foco del teclado.
- El pseudo-elemento `::after` crea el tooltip de las habilidades. `content: attr(data-tooltip)` toma su texto del atributo HTML.
- `@media (max-width: 680px)` cambia el diseno en pantallas estrechas para que las columnas se apilen.
- `prefers-reduced-motion` respeta la preferencia del sistema para reducir las animaciones y el movimiento.

4. JavaScript del navegador

El archivo `js/script.js` obtiene elementos del HTML usando `document.querySelector()`. Actualiza el año del pie de pagina con `new Date().getFullYear()` y controla el formulario.

1. Escucha el evento `submit` del formulario y llama a `preventDefault()` para que la pagina no se recargue.
2. Lee el nombre, correo y mensaje. `trim()` elimina espacios al comienzo y al final.
3. Comprueba que los campos no esten vacios y que el correo tenga un formato basico. Si hay un error, lo muestra y mueve el foco al campo.
4. Deshabilita el boton mientras se envia para evitar envios repetidos y muestra “Enviando mensaje...”.
5. `fetch()` envia una solicitud HTTP a `/api/contact` con metodo `POST`. `JSON.stringify()` convierte los datos a JSON y `Content-Type` declara su formato.
6. Si la respuesta es correcta, limpia el formulario y muestra confirmacion. Si el servidor devuelve un error, presenta su mensaje.
7. `catch` informa problemas de conexion; `finally` vuelve a habilitar el boton aunque el envio haya fallado.
8. Los eventos `input` y `change` limpian el mensaje de estado cuando la persona vuelve a modificar un campo.

> **Validacion en dos lugares:** la validacion del navegador facilita el uso, pero se puede omitir. Por eso el servidor tambien revisa los datos antes de guardarlos.

5. Backend: Node.js y Express

El backend esta en `server.js`. Node.js ejecuta JavaScript fuera del navegador; Express facilita la creacion de rutas HTTP y el envio de respuestas.

- El modulo `node:fs` crea la carpeta que necesita la base; `node:path` construye rutas del sistema.
- `express.json({ limit: "10kb" })` interpreta cuerpos JSON y limita el tamaño de las solicitudes.
- La ruta `POST /api/contact` recibe `nombre`, `correo` y `mensaje`.
- El servidor comprueba que los datos sean textos, quita espacios sobrantes y revisa que los campos sean validos y no superen sus limites.

- `400` indica datos incorrectos; `413`, una solicitud demasiado grande; `201`, que el mensaje se guardo; y `500`, un error interno.
- Las rutas estaticas permiten servir `index.html` y los archivos publicos de `assets`, `js` y `style`.
- El bloque `require.main === module` inicia el servidor cuando ejecutamos el archivo directamente. Permite importar sus funciones en las pruebas sin iniciar otro servidor.

6. Base de datos SQLite

SQLite guarda datos en un archivo local. El proyecto crea `.data/portfolio.sqlite` automaticamente la primera vez que se inicia. No es necesario instalar ni arrancar un servidor de base de datos separado.

La tabla se llama `contact_messages`. El comando SQL que la crea se ejecuta solamente si aun no existe:

```
CREATE TABLE IF NOT EXISTS contact_messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    message TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Columna	Significado
`id`	Identificador unico que SQLite genera automaticamente.
`name`	Nombre de la persona que envia el mensaje.
`email`	Correo electronico de contacto.
`message`	Contenido del mensaje.
`created_at`	Fecha y hora que se completan automaticamente.

`NOT NULL` significa que la columna no puede quedar vacia. `PRIMARY KEY` identifica una fila; `AUTOINCREMENT` genera el ID siguiente. `DEFAULT CURRENT_TIMESTAMP` asigna la fecha actual si no se proporciona otra.

El backend prepara una consulta y pasa los valores aparte:

```
INSERT INTO contact_messages (name, email, message)
VALUES (?, ?, ?)
```

Los signos `?` son parametros reemplazados por los valores al ejecutar la consulta. Asi se evita concatenar texto de usuario dentro del SQL. La base queda fuera de Git porque `.data/` esta en `.gitignore`; los mensajes personales no deben subirse como codigo.

Las pruebas usan `:memory:`, una base temporal en memoria, para no guardar datos de prueba en el archivo real.

7. Archivos de configuracion y pruebas

- `package.json` describe el proyecto, requiere Node.js 22.13 o superior, registra Express y define los comandos `start` y `test`.
- `package-lock.json` guarda las versiones exactas de dependencias para que las instalaciones sean reproducibles.
- `.gitignore` excluye `node_modules/` y `.data/` del control de versiones.

- `test/server.test.js` comprueba que la pagina se sirve, que un mensaje valido se guarda y que datos invalidos se rechazan.

8. Comandos del proyecto

En PowerShell, abre la terminal dentro de la carpeta del proyecto:

Comando	Para que sirve
`node --version`	Comprueba la version instalada de Node.js. El proyecto necesita 22.13 o superior.
`npm.cmd install`	Instala las dependencias descritas en el archivo de bloqueo. Se ejecuta al preparar el proyecto o al cambiar dependencias.
`npm.cmd start`	Ejecuta el script `start` de `package.json`, que inicia `server.js`. Despues abre `http://localhost:3000`.
`npm.cmd test`	Ejecuta las pruebas automatizadas de Node.js.
`node --check server.js`	Comprueba la sintaxis de `server.js` sin iniciar el servidor.
`node --check js/script.js`	Comprueba la sintaxis de `script.js`.

Para detener el servidor, vuelve a la terminal y presiona `Ctrl+C`. En este equipo se usa `npm.cmd` porque PowerShell puede bloquear `npm.ps1`; no hace falta cambiar la politica de seguridad.

9. Preguntas que pueden hacerte

¿Que guarda la base de datos?

Guarda los mensajes del formulario: nombre, correo, contenido y fecha. No guarda los proyectos, que siguen escritos directamente en el HTML.

¿Por que elegiste SQLite?

Para un proyecto pequeño es practica: persiste los datos en un archivo local y no necesita administrar un servidor de base de datos aparte.

¿Por que validar tambien en el servidor?

La validacion del navegador puede desactivarse u omitirse. El backend debe validar cualquier dato antes de guardarlo.

¿El formulario envia un correo?

No. Envía el mensaje al backend y lo guarda en SQLite. El proyecto no tiene conectado un servicio de envio de correos.

¿Que significan los signos de pregunta en la consulta SQL?

Son parametros de una consulta preparada. Se sustituyen por los valores del formulario al ejecutar la insercion, sin armar el SQL concatenando esos textos.

¿Que se podria mejorar para poner el sitio en produccion?

Elegir un hosting que conserve el archivo de SQLite o usar una base alojada, proteger y limitar el formulario frente a abuso, y hacer copias de seguridad. Tambien se puede agregar envio de correo si se necesita.

10. Guion corto para la exposicion

> “El proyecto es un portfolio dividido en frontend y backend. El HTML estructura las secciones, el CSS define la apariencia adaptable y JavaScript da interactividad: valida el formulario y manda sus datos como JSON. En el backend uso Node.js con Express. La ruta POST /api/contact vuelve a validar el mensaje y lo guarda en SQLite, una base local en archivo, usando una consulta preparada. Para ejecutar el proyecto instalo dependencias, lo inicio con npm start y pruebo el comportamiento con npm test.”

Consejo: no hace falta memorizar cada linea. Explica el recorrido de un mensaje y para que sirve cada tecnologia; despues usa las preguntas como repaso.